

3-TP2

Dupla: Arthur Bicalho e Fernando Leite Pagani

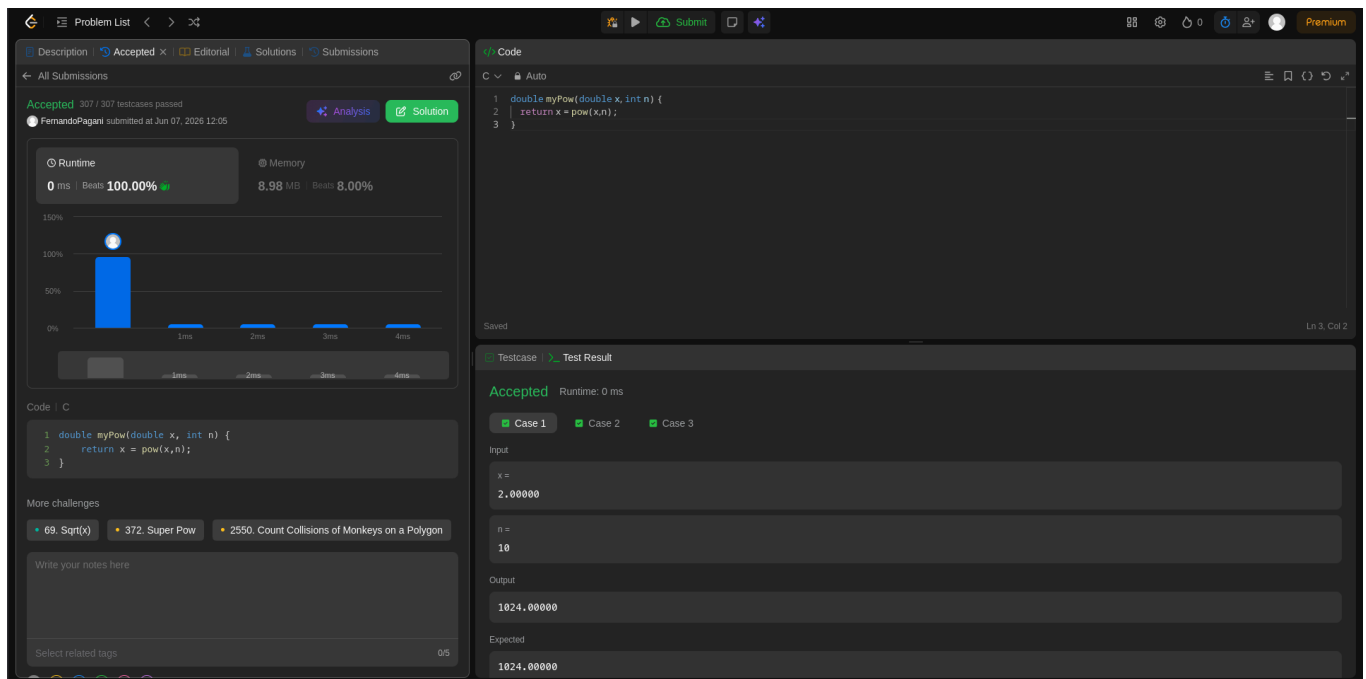
(i) Código de submissão

```
double myPow(double x, int n) {  
    return x = pow(x,n);  
}
```

// vale ressaltar que o leetcode já provê a própria função int main e devidas bibliotecas, logo, somente é necessário criar a função

// Se tentar adicionar próprias bibliotecas e int main função dá erro

(ii) Screenshot de aceite



(iii) Manuscrito

③ LeetCode

Justificativa

A função utilizada é a `pow()`, presente na biblioteca `<math.h>` que já vem incluída pelo próprio LeetCode. A função calcula exatamente x^n para todas as casos cobertos pelas restrições do problema.

As restrições do enunciado garantem que nenhum caso indevido seja testado ~~exatamente~~, portanto a solução cobre todos os domínios de entrada válidos.

Complexidade

A função `pow()` utiliza a "exponenciação rápida" que divide o problema ao meio a cada passo, reduzindo a complexidade de $O(n^2)$ para $O(\log n)$.

Podemos chegar nessa complexidade por meio da pergunta "quantas vezes podemos dividir por 2 até chegar em 0?"

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1 \rightarrow 0$$

$$K \text{ divisões, onde } 2^K = n \text{ e } K = \log_2(n)$$

et complexidade é, portanto:

$$O(\log n) - \text{logarítmica}$$

Tempo de Execução

fórmula operações por segundo:

$$\begin{array}{l} \text{realiza} \\ 3 \text{ operações} \rightarrow 3 \cdot \log_2 n \\ \text{por iteração} \quad \frac{10^8}{10^8} \end{array}$$

para 10^8 operações:

$$\frac{3 \cdot \log_2 10^8}{10^8} \approx \frac{3 \cdot 26,57}{10^8} = 79,71 \cdot 10^{-8} \text{ seg.}$$